

Resumen

En este proyecto se trabaja un problema de predicción del equipo ganador dentro de una partida de League of Legends por medio de IA. Para ello, se usan métodos, estimadores, y parámetros que predican el ganador de dichas partidas. En base a los resultados para cada uno de estos métodos, se determina cual de todos estos es el más óptimo para para lograr predecir las partidas. Y para terminar, se dictarán algunas sugerencias y/o recomendaciones para abordar este problema, o algún problema similar, con mayor precisión y alcance.

Introducción

Entre la gran cantidad de videojuegos han habido algunos que poseen un ambiente competitivo, en los que los jugadores se enfrentan entre sí para demostrar quién es el mejor. Entre esos juegos se encuentra League of Legends, un juego en el que dos equipos de cinco personas se enfrentan para destruir el nexo del enemigo. El equipo ganador es recompensado con puntos para subir de una serie de rangos ordenados de la siguiente manera: hierro, bronce, plata, oro, platino, diamante, maestro, gran maestro y retador. En este proyecto se analiza los resultados obtenidos en una gran cantidad de partidas de jugadores con rango diamante, y con ellos, entrenar una inteligencia artificial que pueda determinar el ganador; la utilidad en esto se encuentra principalmente en las apuestas.

Gracias a que League of Legends es de los juegos más influyentes entre los e-sports, se realizan apuestas en los distintos torneos como si de otro deporte se tratase. Así que, el objetivo principal es ganar la mayor cantidad de apuestas posibles con los resultados que genere la inteligencia artificial. Aunque también se puede utilizar para el balanceo del juego, ya que nos muestra qué tan relevantes son los distintos objetivos de una partida dentro del juego y, a partir de ello, hacer los cambios pertinentes.

Proceso y método

El estudio se hizo a partir de un dataset sobre un historial de partidas de jugadores rango diamante en el videojuego League of Legends; muchos de éstos llegan a rangos superiores. A ese dataset se le aplicó distintos tipos de estimadores, tales como: Decision Tree Classifier, Support Vector Classifier (con kernel RBF) y Multinomial Naive-Bayes, para compararlos y sacar el mejor resultado.

Al iniciar el estudio, se importaron las librerías necesarias y se definieron un par de funciones indispensables para su desarrollo. Posteriormente, se importó el dataset, se comprobó que no existiera redundancia, y, al encontrar redundancia, se eliminaron las columnas que la provocaban. Se utilizó el estimador Decision Tree Classifier

(haciendo uso de distintas profundidades) y el estimador Support Vector Classifier (con kernel RBF).

Al observar los resultados obtenidos con los dos estimadores anteriormente mencionados, se optó por utilizar el estimador Multinomial Naive-Bayes, el cual no trabaja con valores negativos (para evitar errores a la hora de entrenar, se sumó el valor absoluto del menor valor a cada uno de los datos del dataset). Usando varios valores de Alpha (un parámetro del estimador), se buscó determinar el valor cuyo resultado en el estimador sea el mejor. Finalmente, se decidió crear una matriz de confusión con el propósito de ilustrar el desempeño del estimador.



Figura 1. Rango diamante

Resultados

A pesar de ser un problema de naturaleza ligeramente aleatoria, con ayuda del uso de diferentes métodos y técnicas, se mejoró la capacidad del estimador Multinomial Naive-Bayes, al punto de predecir correctamente el resultado de 3 de 4 partidas (Gráfico 2).

Al usar DecisionTreeClassifier y analizar los resultados obtenidos en Train (1.0) y Test (0.6483), se puede ver que presentó OverFitting, por lo que se procedió a comprobar si el dataset utilizado se encontraba balanceado. Al observar que el dataset se encontraba balanceado, se procedió a comprobar por medio de KFold si se podía mejorar el rendimiento del estimador. Notando que la desviación estándar iba aumentando mientras el número de Folds también aumentaba, se optó por usar otro estimador.

Al usar SVC (con Kernel RBF) y la técnica de PCA se pudo observar que la cantidad de componentes usados en PCA no afectó significativamente el rendimiento, por lo que se optó por hacer otra prueba con DecisionTreeClassifier, esta vez, usando diferentes profundidades (Gráfico 1). Después de realizar distintas pruebas, se determinó que no se pueden obtener mejores porcentajes usando este estimador, por lo que las siguientes pruebas se realizaron usando Multinomial Naive-Bayes.

Los resultados obtenidos con este estimador fueron mejores que los obtenidos anteriormente, además de esto, el tiempo de ejecución es mucho menor, por lo que se probó usando distintos valores de Alpha (sin embargo, los valores obtenidos no variaban lo suficiente como para determinar un valor específico de Alpha en donde su resultado fuera el más óptimo).

Gráfico 1. Diagrama de precisión para diferente profundidades con DecisionTreeClassifier

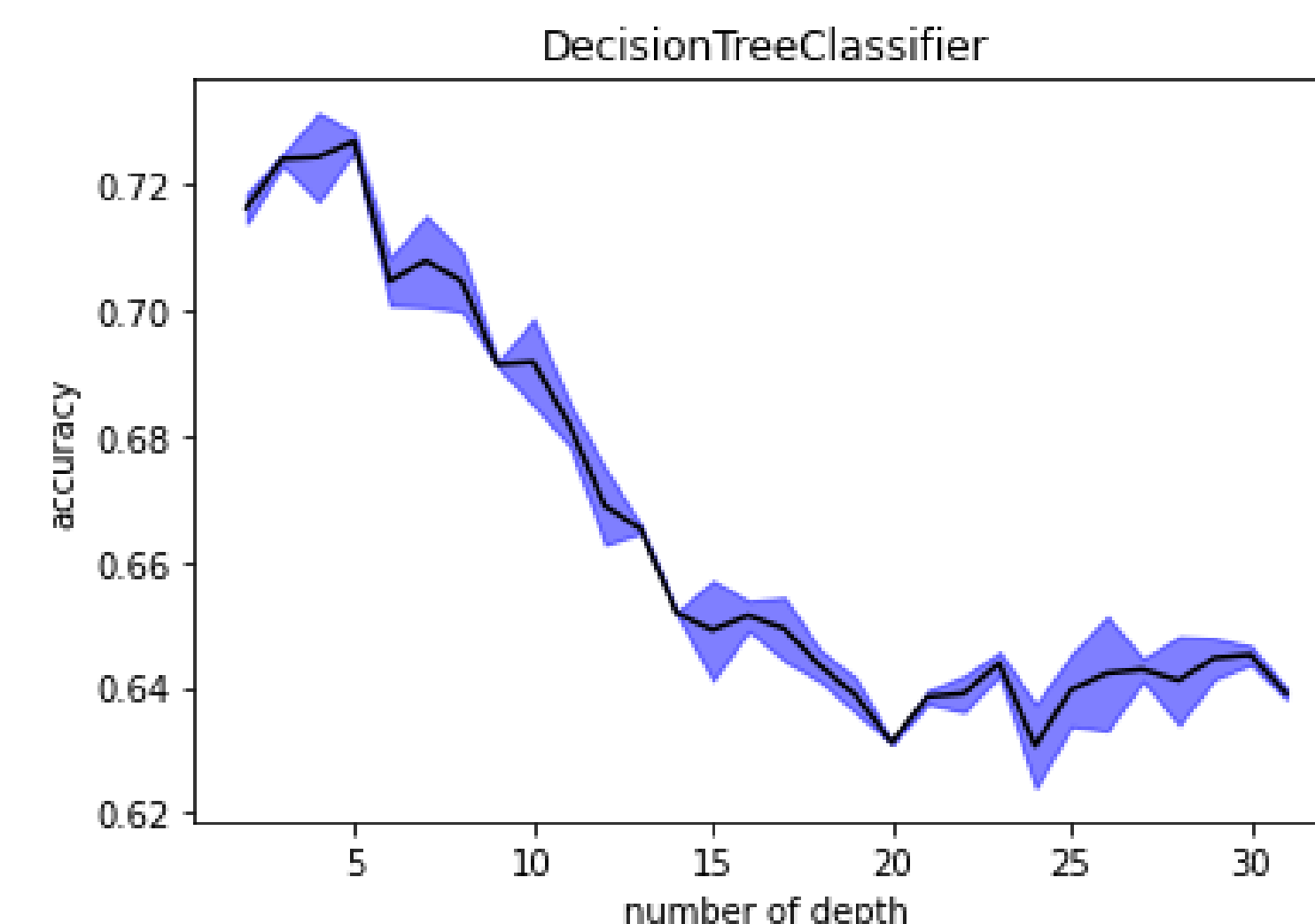
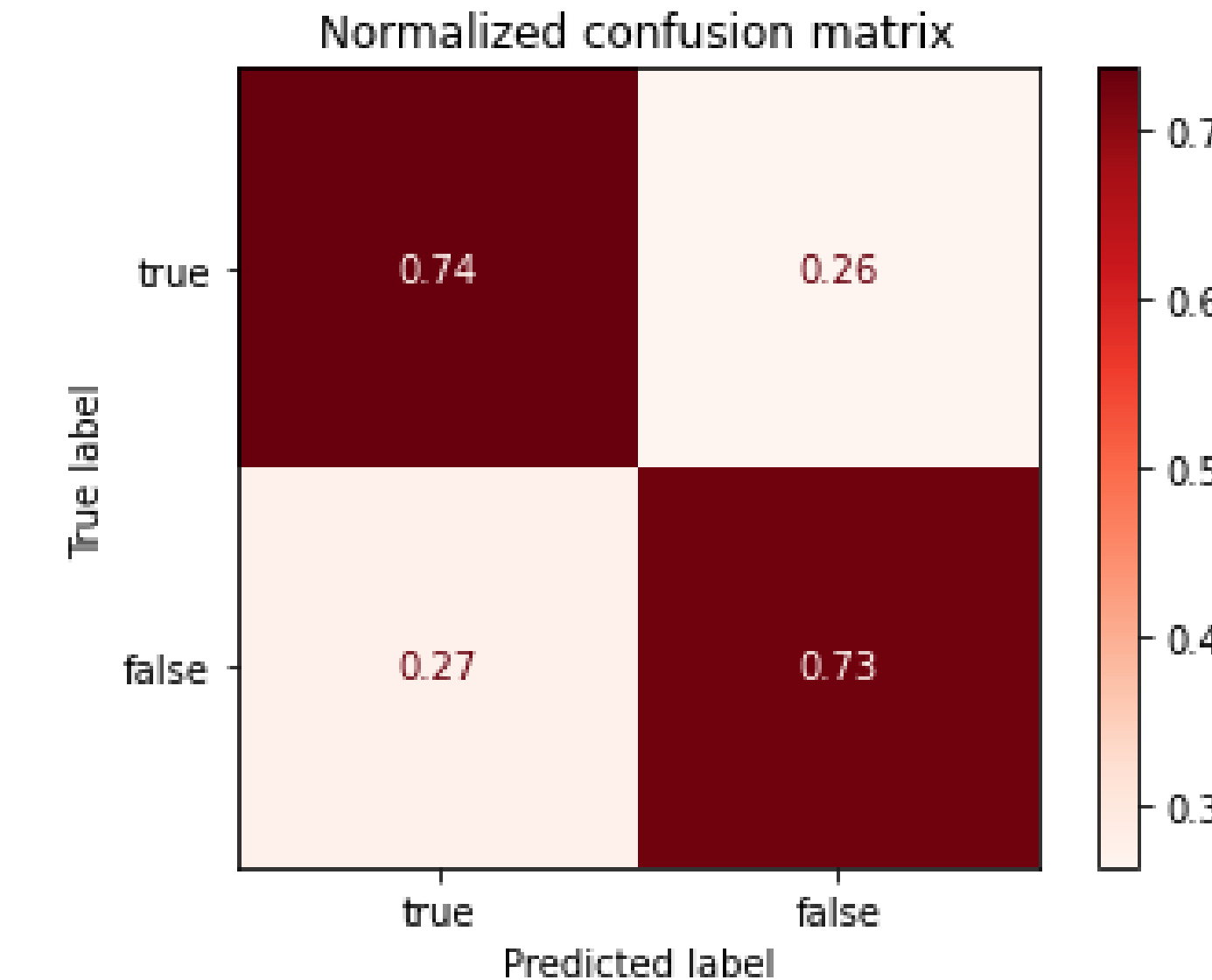


Gráfico 2. Matriz de confusión para MultinomialNB



Conclusiones

- Analizando los resultados obtenidos con cada estimador utilizado, se aprecia que la puntuación es similar (con parámetros óptimos). Este tipo de resultados puede ser una buena señal para indicar que se está cerca del límite de rendimiento al que puede llegar una IA para cierto fenómeno. Sin embargo, si se prueban más estimadores y más parámetros, mejor aproximación se puede tener a unos resultados aún más óptimos.
- El análisis de los resultados para diferentes parámetros con diferentes estimadores es crucial para poder determinar qué estimador es el más apropiado para dar solución al problema planteado.
- Algunos fenómenos son bastante impredecibles, causando que, incluso las mejores inteligencias artificiales con datos óptimos, puedan llegar a equivocarse.
- Algunos estimadores, por más utilizados y complejos que sean, muchas veces no son capaces de otorgar una predicción adecuada al problema planteado.
- Aproximarse a la realidad es complicado; para ello es necesario tener una mayor cantidad de datos que permita a la IA entrenar mejor y disponer de una capacidad de computo que pueda manejar esa cantidad de datos.

Trabajo Futuro

En un futuro se podría trabajar con un dataset enfocado al historial de los resultados en torneos de los equipos profesionales, para obtener resultados más fiables y útiles al momento de apostar, el emparejamiento entre equipos y balanceo de personajes.

Información de contacto

Cristian Guillermo Serrano Acevedo, Email: cristian2170127@correo.uis.edu.co
Joseph Fabian Basto Cuadros, Email: joseph2170081@correo.uis.edu.co
Liliana Paola Castellanos Pinzón, Email: liliana2170095@correo.uis.edu.co

Docente: Gustavo Garzón, gustavo.garzon@saber.uis.edu.co

Referencias Bibliográficas

- League of Legends Diamond Ranked Games (10 min). (13 de Abril de 2020). Recuperado de <https://www.kaggle.com/bobbyscience/league-of-legends-diamond-ranked-games-10-min>
- sklearn.naive_bayes.MultinomialNB. (2007-2020). Recuperado de https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Confusion matrix. (2007-2020). Recuperado de https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html